/ /

Leadizon Jimenez
20251672
Fundamentos de programacion 1

Resumen de los capitulos 1-2-3-4:

Matematica de Computacion

1. Sistema decimal:

El sistema decimal es el sistema numerico que todos usamos en nuestra vida cotidiana. Se llama asi porque esta basado en 10 digitos diferentes del 0 al 9. Este sistema tiene sus raices en la antiguedad, probablemente porque los seres humanos tenemos 10 dedos en las manos, lo que facilito su adopcion universal

Esta forma de presentacion es extraordinariamente eficiente y nos permite escribir numeros muy grandes o muy pequeños de manera compacta. Ademas, las operaciones aritmeticas como, suma, resta, multiplicacion y division se realizan de manera sistematica gracias a esta estructura posicional. El sistema decimal es fundamental para el desarrollo de las matematicas y sigue siendo el estandar en la mayoria de las

Credeal

aplicaciones cotidianas, aunque en computadoras se utilizan otros sistemas mas adecuados para el funcionamiento de los circuitos electricos.

- Sistema Binario, octal, Hexadecimal:

El sistema binario es fundamental en la computadora. Este sistema usa unicamente dos digitos 0 y 1, cada posicion representa una potencia de 2 en lugar de 10.

El sistema binario, el numero 1011 no significa mil once como en decimal sino que representa $1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0$, lo que equivale a 11 en decimal. Aunque se vuelven perfectos para nuestras maquinas, el binario resulta incomodo para nosotros porque los numeros se vuelven muy largo.

Se dio solucion a esto cuando se crearon dos sistemas intermedios: el octal y el hexadecimal: El sistema octal usa ocho digitos (0-7) y agrupa los bits en de tres en tres, mientras que el hexadecimal usa 16 simbolos (0-9 y A-F, donde A = 10, B = 11, C = 12, D = 13, E = 14, F = 15) y agrupa los bits de cuatro en cuatro. Estos sistemas permiten representar numeros binarios de forma mas compacta.

Credeal

-Generalizacion de las conversiones:

Para convertir un numero desde cualquier decimal base, aplicamos lo siguiente: Multiplicamos cada digito para la base elevada a la posicion que ocupa y sumamos todos los resultados.

La conversion inversa, de decimal a cualquier otra base requiere un enfoque diferente. El metodo mas usado es el divisiones sucesivas. Dividimos el numero decimal entre la base objetivo, guardamos el residuo y luego dividimos el cociente nuevamente entre la base. Repetimos este proceso hasta que la base sea cero, y los residuos, leidos de abajo hacia arriba, forman el numero en la nueva base. Por ejemplo: para convertir 25 decimal a binario dividimos 25÷2=12 resto 1, 12÷2=6 resto 0, 6÷2=3 resto 0, 3÷2=1 resto 1, 1÷2=0 resto 1. ~~12~~ Leyendo los residuos de abajo hacia arriba obtenemos 11001.

Para convertir de binario a octal, se agrupan los bits de 3 en 3 desde la derecha y cada grupo se convierte directamente a un digito octal. De manera similar, para convertir hexadecimal agrupamos de 4 en 4.

/ /

-Operaciones Basicas

La suma binaria, las reglas son sencillas al igual que la division, resta y multiplicacion. Se aplican de forma identica en cualquier sistema posicional. El unico factor variable es la base ( ), que determinan cuantos simbolos existen y cuando se genera un acarreo.

Para realizar cualquier (conversion*) operacion todos los operandos deben estar en la misma base.

Si los numeros estan en bases distintas es obligatorio realizar una conversion previa.

Lo que debemos entender es que la matematica no cambia mas bien sus reglas lo unico que cambia es el tamaño de nuestra caja de herramientas (la base). si sabemos sumar en decimal, temporalmente sabemos sumar en binario, octal, hexadecimal, solo se debe aprender a llevar numeros cuando pasamos del limite.

/ /

7. Suma de Dos Cantidades en Complemento a2

En las computadoras, los numeros negativos no se pueden representar simplemente poniendo el signo (-) delante porque toda la información debe codificarse en bits. El metodo mas comun para presentar numeros negativos es el complemento a2, que tiene varias ventajas sobre otros metodos con el signo-magnitud o el complemento a1.

El complemento a2 de un numero binario se obtiene de la siguiente manera primero invertimos todos los bits (cambiamos 0s por 1s y 1s por 0s) luego sumamos el 1 al resultado. Por ejemplo 00000101 invertimos los bits 11111010. Sumamos 1: 11111011: Este es el complemento a2 de 5 es decir -5.

La ventaja del complemento a2 es que podemos sumar numeros positivos y negativos usando el mismo circuito de suma, sin necesidad de verificar signos o restar.
Este metodo tambien tiene la ventaja de tener una unica representacion del cero (0000000) a diferencia del complemento a1 que tiene un cero + y un cero -.

- Aplicaciones de los Sistemas Numericos.

Los sistemas numericos no son solo conceptos teoricos: tienen aplicaciones practicas y directas en casi todos los aspectos de la computacion moderna. Desde el nivel mas bajo de hardware hasta las aplicaciones de alto nivel, estos sistemas estan presentes en el funcionamiento diario de la tecnologia.

El sistema hexadecimal es extremadamente comun en programacion. Los colores en diseño Web especifican en hexadecimal (#RRGGB) donde cada par de digitos ha intensidad de rojo, verde o azul. Las direcciones de memoria se muestran tipicamente en hexadecimal porque es mas compacto que binario pero mantiene una correspondencia directa con el. Los codigos de error los debuggers es unicos, y muchos otros valores en programacion se expresan en hexadecimal.
Cualquier programador que haya hecho debugging o trabajado con herramientas de bajo nivel ha visto valores hexadecimales.

- Principio Fundamental de Conteo

Contar parece una tarea simple, pero cuando enfrentamos problemas mas complejos necesitamos metodos sistematicos. Los principios del conteo nos proporcionan herramientas poderosas para determinar cuantas maneras diferentes hay de realizar una tarea o cuantos elementos hay en conjunto sin tener que enumerarlos todos.

El principio de la multiplicacion es uno de los conceptos mas importantes. Se dice que si una tarea puede dividirse, es una secuencia de pasos, y el primer paso puede realizarse de n_1 maneras, el segundo de n_2 maneras y asi sucesivamente, entonces el numero total de formas de completar toda la tarea es $n_1 \times n_2 \times \ldots$

El principio de la suma complementa al anterior. Se aplica cuando tenemos varias formas mutuamente excluyentes de realizar una tarea. Si se puede hacer algo de n_1 maneras o de n_2 maneras (pero ambas simultaneamente, entonces hay en $n_1 + n_2$ formas totales de hacerlo.

Credeal

- Permutaciones:

Las permutaciones se refieren a los arreglos ordenados de elementos. La palabra clave aquí es "ordenados": en una permutación el orden importa.

Cuando n objetos diferentes y queremos saber de cuantas formas podemos ordenarlos, la respuesta es n!. Esto significa $n \times (n-1) \times (n-2) \times \ldots \times 2 \times 1$ ¿Porque? porque para la primera posición tenemos n opciones, para la segunda quedan n-1 opciones para la tercera n-2 y así sucesivamente. Si tenemos 5 libros y queremos ordenarlo en un estante hay $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$ formas diferente de hacerlo.

En computación, las permutaciones ocurren constantemente. Los algoritmos de ordenamiento reorganizan elementos en diferentes permutaciones en criptografía, podemos decir que las permutaciones se usan para cifrar información. En pruebas de software, a veces necesitamos considerar todas las permutaciones posibles de entradas para asegurar que un programa funciona correctamente.

- Combinaciones:

A diferencia de las permutaciones, el orden no importa en las combinaciones. Cuando seleccionamos un grupo de elementos y solo nos interesa que elementos estan en el grupo, no en que orden estamos hablando de combinaciones. Por ejemplo si digo unas fruta en una canasta seleccionar manzana-guineo-naranja, es lo mismo que seleccionar naranja-manzana-guineo.

En programacion y ciencias de la computacion las combinaciones son fundamentales. En analisis de algoritmos de busqueda, a veces necesitamos evaluar todas las combinaciones posibles parametros. En aprendizaje automatico, la seleccion de caracteristicas involucra elegir combinaciones de variables. Entender cuando usar la diferencia de las permutaciones (orden importa) y cuando usar combinaciones (orden no importa) es crucial para entender, para modelar correctamente problemas del mundo real.

- Aplicacion en la computacion.

Las tecnicas de conteo tienen aplicaciones directas y practicas en casi todas las areas de la informatica. No son solo ejercicios matematicos abstractos sino herramientas esenciales para resolver problemas reales que enfrentan los profesionales de la computacion todos los dias.

En analisis de algoritmos, las tecnicas de conteo nos permiten determinar la complejidad temporal y espacial. Cuando analizamos cuantas operaciones realiza un algoritmo, a menudo contamos permutaciones o combinaciones de datos de entrada. Por ejemplo un algoritmo de ordenamiento por burbuja realiza comparaciones cuyo numero esta relacionado con las permutaciones posibles de los datos. Los algoritmos de busqueda exhaustiva generan y evaluan combinaciones de soluciones potenciales. Entender tecnicas de conteo nos ayuda a precisar si un conteo, algoritmo será practico o si tardara demasiado tiempo para conjuntos de datos grandes.

Concepto de conjunto.

En terminos simples un conjunto es una colección de objetos distintos, considerados con un todo. Los objetos que forman un conjunto se llaman elementos o miembro del conjunto. Los objetos que forman un conjunto que los hace especial es que cada elemento aparece una sola vez y no importa en el orden en que los listemos.

Los conjuntos se representan típicamente con letras mayusculas como A, B, C, y sus elementos se encierran entre llaves. Un ejemplo seria $A = \{1, 2, 3, 4, 5\}$ es el conjunto de los primeros cinco numeros naturales. Si un elemento x pertenece a un conjunto A, escribimos $x \in A$ (se lee "x pertenece a A"). Si no pertene, escribimos $x \notin A$. La caracteristicas de cada elemento aparece solo una vez significa que $\{1, 2, 5, 3\}$ y $\{1, 2, 3, 2, 1\}$ son el mismo conjunto

Algunos conjuntos especiales tienen nombres: el conjunto vacio, denotado $\emptyset$ o $\{\}$, es el conjunto sin elementos; el conjunto universal U tiene todos los elementos bajo consideracion en un contexto particular.

- Subconjuntos:

Los subconjuntos aparecen constantemente en programacion y ciencia de la computacion. Cuando diseñamos una base de datos, las relaciones entre tablas a menudo involucran subconjuntos: una tabla de clientes premium es un subconjunto de todos los clientes. En algoritmos de busqueda y optimizacion a menudo exploramos subconjuntos de soluciones posibles. El problema de la mochila, por ejemplo buscar el mejor subconjunto de objetos que maximiza el valor sin exceder el peso maximo. En grafos, un subgrafo es esencialmente un subconjunto de los vertices y aristas del grafo original. En aprendizaje automatico, la seleccion de caracteristicas busca el mejor subconjunto de variables para entrenar un modelo. El concepto de subconjunto tambien es fundamental para entender relaciones de herencia en programacion orientada a objetos y jerarquias de tipos en sistemas de tipos.

/ /

- Operaciones y Leyes de Conjuntos:

Las operaciones basicas entre conjunto son analogas a las operaciones aritmetica con numeros, pero con sus propias reglas y propiedades. Estas operaciones nos permiten construir nuevos conjuntos a partir de conjunto existente de manera sistematica.

La union de dos conjuntos A y B denotada $A \cup B$, es el conjunto que contiene todos los elementos que estan en A, en B, o en ambos. Un ejemplo seria si $A = \{1,2,3\}$ y $B = \{3,4,5\}$ entonces $A \cup B = \{1,2,3,4,5\}$. La intersección denotada $A \cap B$, contiene solo los elementos que estan en ambos conjuntos simultaneamente $A \cap B = \{3\}$. El complemento de A respecto al conjunto universal U, denotado A' o A^c contiene todos los elementos de U que no estan en A. La diferencia $A - B$ (o $A \setminus B$) contiene los elementos en A pero no en B: $A - B = \{1,2\}$.

Estas leyes son fundamentales en optimización de consultas de bases de datos, simplificación de expresiones logicas en programación, diseño de algoritmos de busqueda, muchas otras aplicaciones.

- Simplificación de expresiones usando Leyes de conjunto:

La simplificación de expresiones de conjuntos es similar a simplificación algebraica, pero usando las leyes específicas de la teoría de conjuntos. Esta habilidad es crucial para optimizar tanto expresiones matemáticas teóricas como código en aplicaciones prácticas.

Supongamos que tenemos la expresión $(A \cup B) \cap (A \cup C)$. Podemos simplificarla usando la ley distributiva: $(A \cup C) \cap (A \cup C) = A \cup B \cap C$. Esta forma simplificada puede ser más eficiente de evaluar computacionalmente.

Cuando optimizamos o escribimos condiciones booleanas complejas en código, aplicar estas leyes puede hacer el código más legible y eficiente. Por ejemplo la condición: $(a \parallel b)$ es equivalente a $!a \&\& !b$ (ley de Morgan). Los optimizadores de consulta en sistemas de bases datos tienen reglas incorporadas basadas en estas leyes para reescribir consultas en formas más eficiente.

/ /

- Relacion entre Teoria de Conjuntos, Logica Matematica y Algebra Booleana.

En computacion, esta relacion tripartita es fundamental. Los circuitos digitales se usan utilizando algebra booleana, donde cada señal es 0 o 1. El comportamiento de estos circuitos puede realizarse usando logica proposicional. Y cuando se trabaja con colecciones de datos de programacion, se usan operadores de conjuntos. Un programador que entienda esta conexion puede aplicar tecnica de un dominio para resolver problemas en otro. Pondre como ejemplo, las tablas de verdad de logica pueden usarse para verificarse identidades de conjuntos. Las tecnicas de minimizacion de expresiones booleanas (Como mapas de Karnaugh) pueden aplicarse para simplificarse consultas de base de datos. Esta unificacion conceptual es una de las ideas mas poderosas en ciencias de la computacion, conectando el hardware fisico (Circuitos) con el software abstracto (Programas y estructuras de datos.

- Conjuntos finitos:

En programación y base de datos, se trabaja constantemente con conjuntos finitos. Una base de datos es un conjunto finito de registro.
Cuando realizamos una consulta SQL, estamos efectivamente calculando la cardinalidad de intersecciones o productos cartesianos. El principio de inclusión-exclusión aparecen en consultas que usan Union, en análisis de algoritmos, la eficiencia a menudo depende del tamaño de los conjuntos involucrados: un algoritmo que itera sobre todos los subconjuntos tendrá complejidad exponencial $O(2^n)$ porque hay 2^n subconjuntos. En estructura de datos como árboles y grafos contamos nodos y aristas, que forman conjuntos finitos. Entender las propiedades de conjuntos finitos es esencial para estimar el uso de memoria y el tiempo de ejecución de programas.

/ /

- Aplicación de Teoría de conjuntos:

La teoría de conjuntos no es solo un fundamento matemático abstracto; es una herramienta práctica que permea casi todos los aspectos de la informática moderna. Desde el diseño de base de datos hasta el análisis de algoritmos, los conceptos de conjunto proporcionan un lenguaje preciso y poderoso para razonar sobre colecciones de datos.

En bases de datos relacionales, la conexión con teorías de conjuntos es explícita.

En análisis de algoritmos y complejidad computacional, la teoría de conjuntos proporciona el marco para definir problemas. El problema Sat se formula como: dado un conjunto de cláusulas ¿existe una asignación de valores de verdad que satisfaga todas? El problema de cobertura de conjuntos busca el menor número de subconjuntos de una colección que cubren todos los elementos. Muchos problemas NP-Completos se definen naturalmente en términos de conjuntos.

/ /

-Proposiciones

Las proposiciones pueden ser simple o compuestas. Las proposiciones simple atomicas no pueden descomponerse en proposiciones mas pequeñas. Las proposiciones compuestas estan formadas combinando proposiciones simples usando conectivos logicos. Por ejemplo si P= "Esta Moviendo" y Q= "Hace frio" podemos formar proposiciones compuesta como "Esta Moviendo y hace frio" o "Si esta Moviendo, entonces hace frio.

En programación, las proposiciones aparecen como expresiones booleanas. La condicion en un if statement es una proposicion: if (x > 0) evalua la proposicion "x es mayor que cero. Las variables booleanas almacenan valores de verdad. Las expresiones con operadores de comparacion ("==", "!=", <, >) son proposiciones. Los sistemas de tipos en programacion pueden verse como sistemas proposicionales: el compilador verifica proposiciones sobre tipos. En especificacion formal de software las precondiciones y postcondiciones de funciones son mas proposiciones que deben ser verdaderas antes y despues de la ejecucion.

/ /

-Tabla de Verdad.

Para hacer una tabla de verdad, listamos todas las combinaciones posibles de valores de verdad para las expresiones simples. Si tenemos 3 proposiciones simples habrá 2^n filas, calculamos el valor de verdad de la expresión compuesta. Por ejemplo, para $(p \wedge q)$ $\vee r$ con tres proposiciones podemos tener $2^3 = 8$ filas. La tabla de verdad nos permite verificar si dos expresiones son lógicamente equivalentes (tienen la misma columna de resultados), identificar tautologías (siempre verdaderas), contradicciones (siempre falsas) y contingencias (a veces verdadera, a veces falsa).

En programación, las tabla de verdad ayudan a verificar la corrección de condiciones ~~simples~~ complejas. Los debuggers modernos pueden generar tablas de verdad para expresiones booleanas. En inteligencia artificial especialmente en sistemas expertos y razonamiento automatizado, las tablas de verdad son usadas para validar reglas lógicas.

/ /

- Inferencia logica:

La inferencia logica es el proceso de derivar conclusiones a partir de premisas usando reglas validas de razonamiento. Es la base pensamiento deductivo y es fundamental tanto en matematica como en computacion.

Las reglas de inferencia mas comunes incluyen Modus Ponens (si $P \to q$ es verdadero y P es verdadero, entonces q es verdadero) Modus Tollens (si $P \to q$ es verdadero q es falso, entonces P es falso), Silogismo Hipotetico (si P es $\to q$ y $\to q \to r$ son verdaderos, entonces $P \to r$ son verdadero), Simplificacion (si $P \wedge q$ es verdadero entonces P es verdadero), Adicion (si P es verdadero entonces $P \vee q$ es verdadero) y resolucion (si $P \vee q$ y $\neg P \vee r$ son verdaderos entonces $q \vee r$ es verdadero).

Estas cadenas nos permiten construir cadenas de razonamiento. Por ejemplo "si estudias aprobaras el examen ($P \to q$). "Estudiaste (P) por modus ponens concluimos aprobaras el examen" (q).

/ /

- Equivalencia Logica.

En programacion, estas equivalencia tienen aplicaciones inmediatas. Cuando escribimos condiciones en codigo aplicar equivalencia puede simplificar la logica. La condicion !(a && b) es equivalencia a !a || b (Morgan) que puede ser mas claro o eficiente dependiendo del contexto. Los compiladores optimizan codigo aplicando estas equivalencias automaticamente. En diseño de circuitos minimizar expresiones booleanas usando equivalencia logica reduce el numero de compuertas, ahorrando espacio en el chip y reduciendo el consumo de energia. En bases de datos el optimizador de consultas reescribe condiciones WHERE usando equivalencia para encontrar planes de ejecucion mas eficiente. En verificacion de software demostrar que dos piezas de codigo son equivalentes a menudo se reduce a demostrar equivalencias logicas entre sus especificaciones. Los refactoring tools IDEs modernos usan equivalencia logica para transformar codigo de manera segura.

/ /

- Argumentos Validos y no Validos.

Es importante distinguir entre validez y solidez. Un argumento valido puede tener premisas falsas. Un argumento solido si es es valido y toda sus premisas son verdaderas. Por ejemplo: "todos los mamiferos ponen huevo falso. Las aves son mamiferos verdaderas por lo tanto las aves ponen huevos falsos. Este argumento es valido en forma pero no solido porque tiene una premisa falsa.

Para determinar si un argumento es valido podemos usar varios metodos. El metodo de tablas de verdad construye una tabla para todas las premisas y la conclusion son verdaderas y la conclusion es falsa. Si existe tal caso, el argumento es invalido. Otro metodo es intentar construir la una derivacion usando reglas de inferencia como vemos ante. Si podemos derivar la conclusion de las premisas el argumento es valido.

/ /

- Demostración Formal:

Una demostración o prueba formal es una secuencia de proposiciones donde cada proposición es una secuencia dada de la premisa o se deriva de proposiciones anteriores usando reglas de inferencia válidas. La última proposición de secuencia es la conclusión que queremos demostrar, las demostraciones formales son el gold standard de certeza en matemáticas y tienen aplicaciones importantes en computación.

Existen varios metodos de demostración. La demostración directa asume las hipótesis y deriva la conclusión paso por paso. La demostración por contradicción, probando así que la conclusión debe ser verdadera. La demostración por contraposición usando el hecho de que $P \to Q$ es equivalente a $Q \to P$, así que en lugar de demostrar $P \to Q$ directa, demostramos $Q \to P$. La demostración por casos divide problemas en casos exhaustivo y mutuamente excluyente, y demuestra la conclusión para cada caso

/ /

- Predicados y sus valores de Verdad:

Un predicado es una proposición que contiene variables y se convierte en una proposición cuando se asignan valores a esas variables. Por ejemplo $P(x)$ = "x es un numero par" es un predicado. $P(4)$ es verdadero $P(5)$ es falso. Los predicados nos permiten expresar proposiciones sobre colecciones o dominio de objetos de manera concisa.

Los cuantificadores nos permiten hacer afirmaciones sobre todos o algunos elementos de un dominio. El cuantificador universal $\forall$ (para todo) afirma que el predicado es verdadero para todos los elementos del dominio por ejemplo $\forall x\,(x^2 \geq 0)$ dice para todo x, x al cuadrado es mayor o igual a cero. El cuantificador existencial $\exists$ (existe) afirma que el predicado es verdadero para al menos un elemento del dominio. Por ejemplo $\exists x\,(x^2 = 4)$ dice "existe un x tal que x al cuadrado es 4" (verdadero: $x = 2$ o $x = -2$).

/ /

-Inducción matemática

El enunciado de la inducción funciona de la siguiente manera: Para demostrar que una proposición $P(n)$ es verdadera para todo numero natural $n \geq n_0$, demostramos dos cosas 1º el caso base $P(n_0)$ es verdadero. 2º paso inductivo para todo $k \geq n_0$ si $P(k)$ es verdadero (hipotesis inductiva), entonces $P(k+1)$ es verdadera. Si demostramos ambas cosas concluimos con $P(n)$ es verdadero para todo $n \geq n_0$. Es como una fila de fichas de dominó: si la primera cae (1º caso base) cada ficha hace caer la siguiente (paso inductivo) todas las fichas caeran.

Por ejemplo demostremos que la suma $1+2+3+\ldots+n = n(n+1)/2$ para todo $n \geq 1$.
1º caso base $(n=1)$: $1 = 1(1+1)/2 = 1$. Verdadero
Paso inductivo $1+2\ldots+k = (k+1)/2$.